

Hit-Zone Aware Embedding Models for Fine-Grained Visual Homoglyph Detection

Hailey Boriel

Department of Electrical Engineering and Computer Science
Massachusetts Institute of Technology
Cambridge, MA 02139
hboriel@mit.edu

May 2026

Abstract

Homoglyph attacks deceive users by substituting visually similar characters in domain or business names, creating spoofs that are indistinguishable to the human eye. String-distance and text-embedding methods are insufficient because they operate on symbolic representations rather than rendered visual appearance. This thesis presents a sequential visual encoding pipeline in which each name string is rendered as a fixed-height grayscale image, decomposed into narrow vertical column slices, and encoded through a lightweight Conv1D stem followed by one of three sequential encoder variants—Conv1D, bidirectional LSTM, or Transformer with Rotary Position Embeddings—trained end-to-end in a Siamese framework. On the Woodbridge benchmark, the Transformer achieves a test ROC-AUC of 0.9900, the BiLSTM 0.9844, and the Conv1D baseline 0.9677, each surpassing the best string-distance baseline (Damerau-Levenshtein, 0.8869) by over eight percentage points and the semantic embedding control (fastText, 0.6519) by over thirty points.

1. Introduction

Business-name spoofing through homoglyph attacks has become a persistent and accelerating threat within the broader phishing landscape. The Anti-Phishing Working Group recorded 1,003,924 phishing attacks in the first quarter of 2025 alone, with financial institutions, software providers, and payment processors collectively accounting for nearly half of all targeted organizations [1]. Homoglyph attacks differ from conventional phishing, exploiting visually similarity rather than social engineering or technical exploitation. A single Unicode replacement, like a Cyrillic *а* for a Latin *a*, or a digit 0 for the letter *o*, is sufficient to create an identifier that appears identical to the human eye while representing an entirely different byte sequence. The effectiveness of this technique is grounded in fundamental properties of human visual perception. A landmark usability study by Dhamija, Tygar, and Hearst found that well-crafted phishing pages fooled more than

90% of participants even when explicitly primed to look for fraud, and that 23% of participants never consulted the address bar or any browser security indicator, relying entirely on visual appearance [2]. The scale of the Unicode character space makes this vulnerability formidable in practice. Roshanbin and Miller conducted a systematic analysis that demonstrated that even restricting attention to the homoglyphs of a single Latin letter, the string `www.google.com` alone can be rendered in more than 1,360 visually deceptive ways [3]. Together, these findings establish that effective detection systems must focus on rendered visual appearance rather than their symbolic representation as codepoints or byte sequences.

Traditional homoglyph and typosquatting detection methods are largely based on string edit-distance metrics, such as Levenshtein and Damerau-Levenshtein distance. These metrics are fast and interpretable, but treat all character substitutions equally, regardless of visual similarity [4, 5]. The most sophisticated extension within this paradigm, TypoPegging proposed by Liu et al., improves this approach by incorporating position-weighted costs and a manually curated visual confusion matrix, achieving strong performance on typo-domain detection [6]. However, its effectiveness is limited by its predefined confusion table. In contrast, Woodbridge et al. showed that a Siamese CNN trained on rendered domain-name images outperformed both standard and visually-enhanced edit-distance methods, demonstrating the advantage of learned visual representations for detecting deceptive character substitutions [7]. At the other extreme, text embedding models such as Word2Vec and BERT are designed to capture semantic rather than visual similarity making them poorly suited for detecting short strings, rare Unicode characters, and visually deceptive substitutions [8, 9]. These limitations suggest that robust homoglyph detection requires direct modeling of visual appearance rather than symbolic or semantic representations alone.

Image-based approaches correctly identify visual similarity as the target property. Woodbridge et al.’s Siamese CNN and GlyphNet proposed by Gupta, Tomar, and Garg encode names as images and have shown strong detection performance [7, 11]. However both encode each name as a single fixed-size two-dimensional image, making detections opaque and difficult to audit as they have no mechanism for localizing where the discrepancy occurred. Ginsberg and Yu proposed the closest precursor to spatially grounded glyph comparison, representing each individual glyph as a binary zone map and computing similarity over spatially aligned zones [12]. While their approach introduces localized visual evidence, it operates only on the glyph level, relies on hand-crafted features, and does not scale to full-name comparison.

This paper addresses the remaining gap through a sequential visual encoding pipeline that renders each name as a grayscale image, decomposes it into a sequence of narrow vertical slices ordered left to right, and encodes that sequence through a lightweight convolutional stem followed by a sequential encoder trained end-to-end. We make two concrete contributions: (1) a sequential visual encoding pipeline that treats rendered name images as ordered slice sequences, enabling position-aware homoglyph embedding; and (2) a systematic comparison of three sequential encoder architectures (Conv1D, BiLSTM, and Transformer) evaluated against string-based and text-embedding baselines on the Woodbridge homoglyph detection dataset.

2. Related Work

2.1. String Similarity Methods

The dominant baseline for detecting homoglyph attacks and typosquatting is string edit distance metrics, most notably the Levenshtein distance [4]. Levenshtein distance measures the minimum number of single-character insertions, deletions, and substitutions to transform one string into another. While widely used for its formal metric properties, it is limited by its flat-cost assumption and therefore treats visually identical and visually dissimilar substitutions as the same. As a result, replacing a character with a visually identical homoglyph is penalized no differently than replacing it with a random character.

Bard’s formalization of the Damerau-Levenshtein (DL) distance [5] extends Levenshtein by adding transposition of adjacent characters as a fourth edit operation. The DL metric retains all four metric axioms, and Bard’s error-correction analysis depends on these axioms to guarantee that correction to the nearest dictionary neighbor is unambiguous. Despite this extension, it retains the same flat substitution cost assumption, inheriting the same blindness to visual character structure as the original Levenshtein

metric.

The most sophisticated attempt to address this limitation is TypoPegging, proposed by Liu et al. [6]. TypoPegging replaces Levenshtein’s flat substitution cost with a weighted version modulated by two factors: a position decay parameter that assigns higher weight to substitutions in earlier character positions (reflecting where users concentrate visual attention), and a character visual distance matrix pre-populated with expert-estimated confusion scores between visually similar character pairs. Evaluated on 1,495 real typo domains, the method achieved an F1 of 94.45%. However, the author’s acknowledge that threshold-based detection cannot reliably distinguish intentional spoofs from legitimately similar names registered by brand owners. In addition, TypoPegging’s coverage depends on its manually curated character visual cost matrix, limiting its ability to detect novel Unicode substitutions not anticipated during construction.

Table 1 summarises the core mechanism and primary homoglyph-detection limitation of each approach described above.

2.2. Image-Based Homoglyph Detection

The limitations of string-based methods motivated a shift to purely visual representations of rendered text. The earliest found work is Ginsberg and Yu’s hit-zone framework [12], which renders glyphs as images and encodes them into four binary zone maps at progressively finer spatial granularities. Similarity is then computed over their aligned zone vectors, allowing visually identical cross-script characters to be matched regardless of Unicode representations.

The transition from hand-crafted visual features to learned representations was established by Koch et al. with the Siamese neural network: twin convolutional networks with tied weights that process two input images in parallel, where visually similar images cluster together regardless of their symbolic identities. This approach was adapted by Woodbridge et al. [7] who rendered process and domain names as binary images and trained a lightweight Siamese CNN using contrastive loss. At deployment, all whitelist embeddings are indexed in randomized KD-Trees for approximate nearest-neighbour retrieval to reduce search cost.

Gupta, Tomar, and Garg identified dataset scale and the requirement for paired real/spoof inputs as limitations of the Woodbridge approach [11]. They propose Glyphnet, an attention-based CNN trained on a constructed 4 M-image dataset of rendered domains. By treating homoglyph detection as an image-classification, Glyphnet removes the requirement for paired spoof-legitimate examples during inference. The authors report strong classification performance and find that pretraining has little benefit, suggesting that typographic spoof detection differs substantially

Table 1: Summary of string similarity methods for homoglyph and typosquatting detection, based on information presented in this section.

Method		Core Mechanism	Key Limitation for Homoglyph Detection
Levenshtein distance [4]	dis-	Minimum insertions, deletions, and substitutions required to transform one string into another	Flat substitution cost: visually identical and visually distinct character pairs incur the same unit penalty
Damerau–Levenshtein [5]		Extends Levenshtein distance with transposition of adjacent characters as a fourth edit operation	Retains flat substitution cost; inherits the same blindness to visual character structure
TypoPegging [6]		Position-decayed weighted edit distance with expert-curated visual confusion scores between character pairs	Threshold cannot reliably distinguish intentional spoofs from legitimately similar names; coverage is bounded by the manually curated confusion table, failing on novel Unicode substitutions absent from the table

from conventional image recognition tasks.

Lee et al. [15] addressed the data scarcity problem from the generative side with PhishGAN, a conditional GAN that synthesizes diverse homoglyph images conditioned on text inputs rendered as images. They demonstrated that models trained on synthetic samples can generalize to real attacks.

Across all of these approaches, the image encoding is a fixed-size tensor produced by processing the rendered name as a single image. No method decomposes the name into a positionally ordered sequence of character-level or column-level representations. Consequently, explicit information about position-indexed representation designed for direct substitution localization is architecturally impossible. The model is structurally blind to where in the name the attack occurred. This is the gap that the hit-zone-aware sequential architecture proposed is designed to close.

2.3. Sequential Visual Encoding

The key design decision in the encoding architecture proposed in this paper is decomposing a rendered name image into a positionally ordered sequence of narrow vertical slices and processing that sequence through a bidirectional recurrent or attention-based encoder. This choice is motivated by prior work that established the viability of treating columnar image structure as the primary unit of sequential analysis.

The direct architectural ancestor is the Convolutional Recurrent Neural Network (CRNN) of Shi, Bai, and Yao [16]. The CRNN’s central contribution is a reading strategy that is foundational to the present work: a VGGstyle CNN processes a fixed-height input image and produces a feature map that is read column by column from left to right to

yield an ordered sequence of feature vectors. It encodes the visual content of the character or sub-character region at that horizontal position, preserving left-to-right spatial order by construction. This sequence is then fed into two stacked bidirectional LSTM layers, capturing both left-to-right and right-to-left context. The CRNN demonstrates that horizontal position in the image naturally corresponds to position within a text sequence, making column-wise feature extraction an effective representation for visual text processing. This paper inherits this spatial inductive bias and applies it to a pairwise similarity task.

More recent sequential vision architectures, including Tatsunami et al.’s Sequencer2D, further demonstrate that axis-aligned sequence processing can serve as an effective alternative to purely convolutional or attention-based representations [17]. Similarly, Tsai et al.’s Stripformer demonstrated the utility of strip decomposition in a 2D image processing task that is architecturally distant from text recognition yet structurally parallel in design.

Taken together, these precedents establish a principled and well-validated basis for the slice-based sequential encoding strategy proposed in this paper. Applying these techniques, where each name is decomposed into a sequence of narrow vertical slices and encoding that

sequence through a convolutional stem followed by a bidirectional sequential encoder, to rendered name images for homoglyph detection, is architecturally natural and, to the best of our knowledge, entirely unexplored in the homoglyph detection literature.

2.4. Pairwise Similarity and Metric Learning

The pairwise training strategy and comparison architecture of this paper are grounded in metric learning. Koch,

Zemel, and Salakhutdinov [14] established the canonical formulation for visual verification, in which their Siamese network passes two input images through twin convolutional branches with tied weights. The resulting feature vectors are compared via a learned weighted L1 distance, and produce a binary prediction with cross-entropy loss. Weight tying ensures parameter-efficiency and symmetry such that the network assigns the same score regardless of which image is presented to which branch. Their work also demonstrated how pairwise verification can be used for one-shot classification and retrieval tasks.

Schroff et al.’s FaceNet [19] demonstrated that optimizing a compact embedding end-to-end for a verification objective outperforms repurposing a classification bottleneck, because the classification objective does not directly optimize for the inter-class geometric separation that retrieval requires. The present paper follows this principle by training the Siamese encoder end-to-end with Binary Cross-Entropy loss on pair labels, deferring margin-based contrastive objectives to future work.

2.5. Text Embeddings and Their Limitations for Visual Similarity

The principled alternative to image-based approaches is to represent names directly as text and compute similarity in an embedding space induced by a pretrained language model. However models such as Word2Vec’s Skip-gram model [8] and BERT [9] are optimized to capture semantic similarity rather than visual appearance. As a result, names that appear in similar contexts receive similar embeddings, even when they contain visually deceptive character substitutions. This objective is fundamentally misaligned with homoglyph detection, where the visual form of individual characters is the primary signal of interest.

Taken together, the related work reveals three limitations in the architecture. String-similarity methods lack perceptual grounding and can be circumvented by Unicode substitution characters absent from manually curated confusion tables. Text embedding methods are semantically misaligned: prioritizing semantic similarity rather than rendered appearance. Prior image-based methods improve the representation by grounding similarity judgments in rendered visual appearance, but encode each name as a monolithic fixed-size tensor, providing limited support for substitution localization. This paper addresses all three gaps: a script-agnostic rendering pipeline and a sequential slice encoder that preserves positional structure with tractable substitution localization.

3. Methodology

3.1. Problem Formulation

Let s_a and s_b denote a candidate name and a reference legitimate name, respectively. The task is to predict a binary spoof label $y \in \{0, 1\}$, where $y = 1$ indicates that s_a is a visual spoof of s_b and $y = 0$ indicates that the pair is legitimate. Let $f(\cdot)$ denote the visual encoder, and let $\mathbf{e}_a = f(s_a)$ and $\mathbf{e}_b = f(s_b)$ denote the d -dimensional embeddings. Classification is performed using the cosine similarity between the two embeddings.

This formulation is a Siamese metric learning problem [7, 14]. The encoder weights are shared across both branches, ensuring the similarity function is symmetric: $\text{sim}(f(s_a), f(s_b)) = \text{sim}(f(s_b), f(s_a))$. The encoder is trained on labeled pairs drawn from the dataset, whose columns are `fraudulent_name`, `real_name`, and `label`.

The dataset consists of labeled name pairs $((s_a, s_b))$. Critically, negative pairs (label = 0) are not drawn uniformly from the name space but are specifically constructed from pairs with low but nonzero edit distance. This design ensures that string-distance baselines receive a non-trivial representation in the negative set, making the evaluation substantially harder than one using randomly sampled negatives. The practical consequence is that a model evaluated on this dataset must learn genuinely visual discriminative features rather than string-length or character-frequency artifacts to achieve high performance.

3.2. Visual Rendering Pipeline

Each input name string is converted to a fixed-height grayscale image before neural processing. The renderer uses DejaVu Sans at a fixed height of 32 pixels; while image width scales naturally with string length; no horizontal scaling or cropping is applied. The output is a floating-point array in $[0, 1]$, where 0 represents the black background and values approaching 1 represent foreground text strokes, normalised at render time.

Grayscale rendering is preferred over colour rendering due to the information relevant in a glyph: stroke geometry, curvature, and spatial coverage rather than chromatic variation. Pixel intensities are normalized to $[0, 1]$ at render time, with background set to black (0.0) and foreground strokes producing values approaching white (1.0). Gupta et al. reports that grayscale rendering consistently outperformed colour rendering in their GlyphNet evaluation, attributing the advantage to better preservation of pixel contrast at character boundaries after resizing [11].

Names are rendered as a single image as opposed to isolated character rendering, in which each character is encoded and compared independently. This word-level rendering preserves the full set of perceptual cues and

inter-character relationships that determine how a name appears to a human reader.

No fixed-width padding is applied during rendering; names of different lengths produce images of different widths, and the downstream slicing stage accommodates variable-width inputs by construction.

3.3. Slice-Based Sequential Representation

The central representational innovation of this paper is the conversion of a rendered name image into a positionally ordered sequence of narrow vertical strips. After rendering, the image is divided into column-wise slices of fixed width W pixels with stride S pixels. With W_{img} denoting the rendered image width in pixels, this produces a sequence of N strips (Equation 1), each of shape $H \times W$:

$$N = \left\lfloor \frac{W_{\text{img}} - W}{S} \right\rfloor + 1 \quad (1)$$

With the selected configuration of $W = 6$ and $S = 6$ (non-overlapping), a 32-pixel-tall rendering of a typical six-character name produces approximately 20 strips.

By slicing this undifferentiated tensor becomes a positionally ordered token sequence: each strip has a well-defined horizontal position, and the model can attend to specific elements of the sequence where the visual discrepancy occurs.

Leading and trailing slices that are entirely or nearly entirely white, produced by the font side-bearing whitespace, are retained in the default configuration. The ablation study (Section 3.5) evaluates whether removing these boundary slices improves validation AUC; results show the effect is negligible (0.689 with retention versus 0.686 with removal), so all boundary slices are kept to avoid the added inference-time cost of the removal step. Internal slices that fall between characters are likewise preserved as inter-character spacing is a perceptual feature of the rendered name.

The design space for the slicing parameters is explored in the ablation experiments of Section 4. The slice width W is varied over $\{3, 4, 6, 8, 16, 32\}$ pixels, and the stride S is varied to produce both overlapping ($S < W$) and non-overlapping ($S = W$) configurations. Narrower slices produce longer sequences with finer positional resolution at the cost of smaller per-slice receptive fields; wider slices produce shorter sequences but capture more contextual pixel content per strip.

Figure 1 illustrates this three-stage pipeline for the pair “Google” (legitimate) and “G00gle” (spoof).

3.4. Sequential Encoder Architectures

The encoder has two stages: a shared Conv1D stem that extracts local visual features within each slice, followed

by one of three sequential encoder variants that model dependencies across the ordered slice sequence.

Conv1D Stem. Each slice is passed through a two layer convolutional stem that projects raw slice pixels into the model’s embedding space. The stem applies

$$\begin{aligned} \text{Conv1d}(\text{slice_dim} \rightarrow \text{embed_dim}, k = 3, p = 1) &\rightarrow \text{ReLU} \\ \text{Conv1d}(\text{embed_dim} \rightarrow \text{embed_dim}, k = 3, p = 1) &\rightarrow \text{ReLU} \end{aligned}$$

where $\text{slice_dim} = H \times W = 32 \times 6 = 192$ and $\text{embed_dim} = 128$. Both layers use same padding, preserving sequence length. The kernel spans three consecutive strips, allowing local context to be incorporated before the sequential encoder stage.

Conv1D Baseline. In the simplest configuration, no additional sequence encoder follows the stem. The Conv1D stem output is passed directly to the pooling layer, providing a baseline that captures local context but not long-range dependencies.

Figure 3.2 shows the end-to-end encoding flow for the BiLSTM and Transformer variants, from raw input string to final name embedding.

Conv1D + BiLSTM. A single-layer bidirectional LSTM is applied to the Conv1D stem output. With $\text{embed_dim} = 128$, the LSTM uses $\text{hidden_size} = 64$ per direction. The final forward and backward hidden states are concatenated to produce a 128-dimensional name embedding, eliminating the need for a separate pooling step. This design follows the CNN-BiLSTM pattern introduced by CRNN [16] for visual sequence modelling.

Conv1D + Transformer. Two stacked Transformer encoder layers [26] are applied to the Conv1D stem output, using the pre-norm convention (layer normalisation before each sublayer, which is more stable for training from scratch than post-norm). Multi-head attention uses 4 heads with head dimension 32 ($128/4$) and feed-forward width 256. Unlike the LSTM, self-attention allows direct comparisons between any pair of slice positions.

To address variable-length slice sequences, the Transformer encoder uses Rotary Position Embeddings (RoPE) [28], which encode position by rotating the query and key vectors before computing attention scores. The resulting attention weights depend only on the relative offset between two slice positions rather than absolute position.

Pooling Strategies After the encoder stage, the per-strip sequence (B, N, d) is reduced to a fixed-size name embedding (B, d) by one of four strategies evaluation in the ablation. *Mean pooling* averages across all valid (non-padded) strip positions. *Max pooling* selects the maximum

Table 2: Encoder–pooling compatibility. Checkmarks indicate valid combinations; dashes indicate the strategy is not applicable for that encoder. Information is drawn from the encoder descriptions in this section.

Encoder	Mean	Max	Attention	CLS token
Conv1D stack	✓	✓	✓	–
BiLSTM	–	–	–	–
Transformer	✓	✓	✓	✓

value per embedding dimension. *Attention pooling* learns a single linear scorer over strip representations, computes softmax weights over valid positions, and forms a weighted sum. For the Transformer encoder only, a learnable [CLS] token may be prepended to the slice sequence, with the output used directly as the name embedding, following the aggregation design of BERT [9] and ViT.

The formal definitions of all three learnable pooling operations are collected in Equations (2) - (4). Here $x_i \in \mathbb{R}^d$ denotes the encoder output at valid strip position i , V is the set of valid (non-padded) positions, and $\mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}$ are the learned attention parameters:

$$\mathbf{e}_{\text{mean}} = \frac{1}{|V|} \sum_{i \in V} \mathbf{x}_i \quad (2)$$

$$\mathbf{e}_{\text{max},j} = \max_{i \in V} x_{i,j}, \quad j = 1, \dots, d \quad (3)$$

$$\mathbf{e}_{\text{attn}} = \sum_{i \in V} \alpha_i \mathbf{x}_i, \quad \alpha = \text{softmax}\{\mathbf{w}^\top \mathbf{x}_i + b\}_{i \in V} \quad (4)$$

Table 2 summarises which pooling strategies are valid for each encoder type. The BiLSTM is excluded from all pooling columns because the concatenation of its forward and backward final hidden states already produces a fixed-size name embedding.

3.5. Siamese Training Framework

The detection model is trained as a Siamese network [14]: both input names pass through the same encoder $f(\cdot)$ with shared weights, producing embeddings $\mathbf{e}_a$ and $\mathbf{e}_b$. Weight sharing ensures that the same visual feature pattern triggers the same representation regardless of which branch the input enters. This symmetry ensures that $\text{sim}(f(s_a), f(s_b)) = \text{sim}(f(s_b), f(s_a))$ without any additional constraint.

The two embeddings are passed to a similarity head that computes their cosine similarity and applies a single linear layer to produce a binary logit. The model is trained with Binary Cross-Entropy loss (`BCEWithLogitsLoss`), where a positive label ($y = 1$) indicates a spoof pair and a negative label ($y = 0$) indicates a legitimate pair. At inference time the logit is converted to a probability via sigmoid, with a default threshold of 0.5. BCE loss is well matched to this setting because the task is defined directly in terms

of binary pair labels, and BCE optimises the log-likelihood of the correct assignment at each training step.

All models are trained using the Adam optimiser with learning rate 10^{-3} , no decay schedule, batch size 64, and 20 training epochs. The global random seed is fixed to 7 across PyTorch, NumPy, and Python’s random module, and `torch.nn.deterministic` enabled for reproducibility. The checkpoint achieving the highest validation ROC-AUC is retained as the final model.

3.6. Ablation Study Design

The visual encoding pipeline involves several discrete design choices whose optimal settings cannot be determined analytically: how the variable-length slice sequence is reduced to a fixed-size embedding, whether boundary blank slices should be removed before encoding, at what spatial granularity the name image is sliced, and the rendering polarity of the background. To isolate the effect of each design decision, one parameter was varied at a time while all others were held at their default values. All ablation experiments were conducted using the Conv1D baseline encoder to minimise confounding effects from more complex sequence models.

The study evaluated four factors:

- **Pooling strategy:** mean pooling, max pooling, and attention pooling.
- **Blank-slice removal:** retaining or removing leading and trailing whitespace slices produced by font side-bearings.
- **Slice width and stride:** slice widths ($W \in \{3, 4, 6, 8, 16, 32\}$) pixels and both overlapping ($(S < W)$) and non-overlapping ($(S = W)$) configurations.
- **Rendering polarity:** only black-background rendering was evaluated in this work; alternative polarity settings are noted as a limitation and left for future investigation.

The results of these experiments are presented in Chapter 4 and are used to determine the default preprocessing and pooling configuration for the main encoder comparisons.

4. Experiments and Results

4.1. Experimental Setup

Dataset. All experiments use the domain-name homograph dataset introduced by Woodbridge et al. [7], stored as `domains_spoof.pkl`. Each record consists of a candidate fraudulent name, a reference legitimate name, and

a binary spoof label. Negative pairs (label = 0) are constructed from names with low but nonzero edit distance, ensuring that the model is challenged by legitimate pairs that are already close under edit distance. The dataset is divided into train, validation, and test splits. The test partition contains 256,886 pairs with class proportions consistent across all splits.

Hardware. Experiments were conducted on the MIT OpenMind Research Computing and Data (ORCD) cluster using NVIDIA GPU nodes allocated via SLURM. A global random seed of 7 is fixed for PyTorch, NumPy, and Python’s `random` module at the start of every run, and `cuda.deterministic = True` is set to eliminate non-determinism from cuDNN kernel selection, ensuring that differences between encoder variants reflect architecture. All three encoder variants (Conv1D, BiLSTM, Transformer) are trained under identical conditions.

Shared hyperparameters. Rendering uses DejaVu Sans at a fixed height of 32 px with a black background (foreground strokes approaching 1.0, background 0.0). The preprocessing configuration selected in Section 4.2 fixes slice width $W = 6$ px and non-overlapping stride $S = 6$ px, with boundary blank slices retained. All encoder models share an embedding dimension of $d = 128$.

Evaluation metrics. The primary scalar evaluation metric is the Area Under the Receiver Operating Characteristic Curve (ROC-AUC). Accuracy, precision, recall, F1 score, specificity, Matthews Correlation Coefficient (MCC), and average precision are reported as secondary metrics.

4.2. Preprocessing Sweep

The strip-design sweep trains 74 model configurations (72 in the standard factorial grid plus two supplementary runs) to determine the fixed preprocessing hyperparameters used by all subsequent encoder experiments. Each configuration varies one or more of four design axes while holding all others at defaults. All sweep runs use the Conv1D baseline encoder with `embed_dim = 128`, 20 epochs, and black background. The sweep was conducted without a fixed random seed; because all 74 configurations were run under the same unseeded conditions, differences in validation AUC across configurations remain attributable to the preprocessing variable under study rather than to stochastic variation. Seed 7 was introduced after the sweep and fixed for all subsequent encoder experiments (Conv1D, BiLSTM, and Transformer). During the Transformer pooling exploration, seeds 13 and 19 were briefly tested on the attention-pooling variant and showed only marginal differences from seed 7, confirming that the sharp performance drop observed for attention pooling reflects a structural issue with that

strategy rather than a seed-specific artifact; seed 7 was therefore retained for consistency across all encoder runs.

4.3. Preprocessing Results

Three pooling strategies are evaluated across 24 configurations each: global mean pooling, global max pooling, and learned attention pooling. Mean pooling achieved the strongest overall performance, with the highest marginal mean validation AUC (0.708 ± 0.065 , maximum 0.813). Its performance distribution was also the most concentrated in the upper half of the AUC range, indicating stable performance across a wide range of slice-width and stride combinations. Attention pooling performed competitively but was consistently below mean pooling, while max pooling exhibited substantially higher variance.

Blank-strip removal had no measurable effect on performance. Retaining boundary slices produced a marginal mean AUC of 0.689 compared with 0.686 when removal was applied, while avoiding any additional preprocessing. Since the two conditions are empirically indistinguishable, boundary slices are retained in the default configuration to save computation time.

Slice width and stride were evaluated jointly because their effects depend on each other. Marginally, $W = 16$ px produced the highest average AUC (0.705 ± 0.058), although $W = 6$ px achieved comparable performance. Examination of the width-stride grid showed that the non-overlapping configuration $(W, S) = (6, 6)$ achieved the highest mean AUC overall (0.754) and the lowest variance ($\sigma = 0.018$). While several larger width configurations produced strong individual runs, they substantially reduced the number of sequential tokens available to downstream sequence encoders.

Table 3 summarizes the preprocessing ablation results. The preprocessing pipeline retains boundary slices, applies mean pooling, and uses a non-overlapping slice configuration of $(W, S) = (6, 6)$, which provided the best balance between predictive performance, stability, and positional resolution.

4.4. Encoder Comparison

All three encoder variants are evaluated under the fixed preprocessing configuration selected in Section 4.2: slice width $W = 6$ px, non-overlapping stride $S = 6$ px, mean pooling (at the sweep stage), black background, and 32 px rendering height. Each encoder is trained for 20 epochs with seed 7, Adam optimiser at 10^{-3} , and batch size 64. Test-set metrics are computed at the best-F1 operating threshold identified from the test predictions and at the fixed threshold of 0.5; both are reported for completeness. For the Transformer, a preliminary pooling-strategy selection experiment precedes the full 20-epoch run. Table 4 reports all three encoders on the full set of test-set metrics.

Table 3: Selected preprocessing configuration for all main encoder experiments.

Axis	Options tested	Selected	Rationale
Pooling strategy	mean, max, attention	mean	Highest marginal mean AUC (0.708); lowest variance; most stable across other axes.
Blank-strip removal	keep, remove	keep	Negligible AUC difference ($\Delta = 0.003$); removal adds inference cost with no measurable benefit.
Background polarity	black	black	Only condition evaluated in sweep; white-background results deferred.
Slice width W	3, 4, 6, 8, 16, 32 px	6 px	Strongest (W, S) pairing in sweep (AUC 0.754, $\sigma = 0.018$); thick upper-tail distribution; sufficient positional granularity for sequential encoders.
Stride S	$W, W/2$	W (non-overlapping)	Comparable marginal mean AUC to overlapping; halves sequence length, reducing inference time.

Conv1D Encoder (Baseline) The Conv1D baseline passes the slice sequence through the two-layer convolutional stem followed immediately by global mean pooling, with no recurrent or attention-based sequence modeling. It achieved a best validation ROC-AUC of 0.9888 and a test ROC-AUC of 0.9677, with average precision of 0.9820. At the best-F1 operating threshold ($t = 0.4729$), the model achieved $F1 = 0.9309$, $MCC = 0.8050$, and specificity = 0.8679; at the fixed threshold of 0.5 the corresponding values are $F1 = 0.9303$, $MCC = 0.8055$, and specificity 0.8787. The near-identical results under both thresholds indicate that the model’s sigmoid outputs are well-calibrated around 0.5 and that threshold selection does not materially affect downstream performance. However, as the baseline architecture relies solely on convolutional receptive fields and global pooling, it provides no mechanism for modelling longer-range dependencies across the rendered name, which limits performance relative to the sequential encoders evaluated subsequently.

Bidirectional LSTM Encoder The BiLSTM encoder appends a single bidirectional LSTM layer (hidden size 64 per direction, final hidden states concatenated to a 128-dimensional embedding) to the shared Conv1D stem. The Conv1D+BiLSTM encoder achieved a best validation ROC-AUC of 0.9933 and a test ROC-AUC of 0.9844, with average precision of 0.9906. At the best-F1 threshold ($t = 0.4705$), the model obtained $F1 = 0.9540$, $MCC = 0.8695$, and specificity = 0.9032. Compared with the Conv1D baseline, the BiLSTM improved test ROC-AUC by 0.0167 and increased specificity by 0.0353, indicating a substantial reduction in false-positive classifications. The improvement supports the hypothesis that bidirectional sequence modelling captures discriminative contextual re-

lationships between character regions that are unavailable to local convolutional processing alone.

Transformer Encoder Before the training run, a preliminary experiment is run to select a pooling strategy for the Transformer (global mean, global max, attention pooling, or CLS token). Max pooling achieves the highest validation AUC (0.9919) and is selected for the full experiment. The final Transformer model achieves the highest overall performance, with a best validation ROC-AUC of 0.9900 and average precision of 0.9943. At the best-F1 threshold ($t = 0.4557$) the Transformer achieves $F1 = 0.9637$, $MCC = 0.8980$, and specificity 0.9302. These results suggest that global self-attention provides the most effective representation of the sequential visual structure present in rendered name pairs, allowing the model to exploit long-range dependencies more effectively than either convolutional or recurrent alternatives.

4.5. Baseline Comparison

Table 5 places all six baseline methods (evaluated under identical conditions on the Woodbridge test split) alongside the three proposed encoders. Every visual-sequential model substantially outperforms every directly evaluated baseline by a wide margin on ROC-AUC. Of the string-based baselines, Damerau-Levenshtein performed the strongest, achieving a test ROC-AUC of 0.8869. However, all three proposed encoders outperform it, with Conv1D, BiLSTM, and Transformer models achieving 0.9677, 0.9844, 0.9900. The Transformer improves upon the best string-distance baseline by 10.3 percentage points, while also achieving the highest average precision (0.9943), F1 score (0.9637), and MCC (0.8980). Among the con-

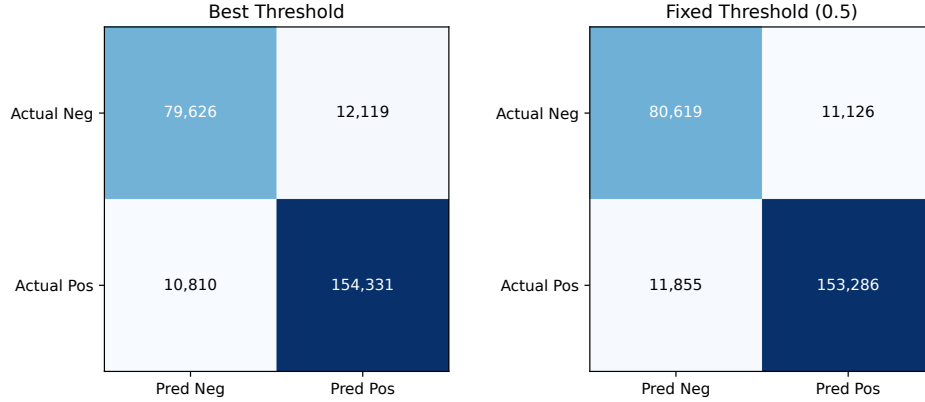


Figure 1: Test-set confusion matrices for the Conv1D encoder at the best-F1 threshold ($t = 0.4729$, left) and at the fixed threshold of 0.5 (right). Counts: true negatives, false positives (top row); false negatives, true positives (bottom row).

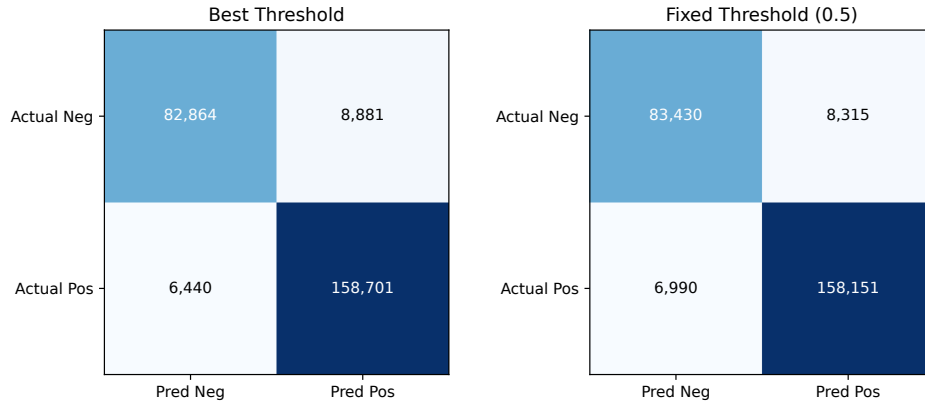


Figure 2: Test-set confusion matrices for the Conv1D+BiLSTM encoder at the best-F1 threshold ($t = 0.4705$, left) and at 0.5 (right).

ventional approaches, the string-distance methods form a relatively tight performance cluster (ROC-AUC 0.8425 - 0.8869), indicating that edit-distance information remains a strong signal for homoglyph detection. However, the limited improvement provided by TypoPegging’s [6] visual-confusion weighting suggests that character-level visual priors alone are insufficient when embedded within a fundamentally string-based framework. The semantic embedding baseline (fastText) performs substantially worse (ROC-AUC 0.6519) and collapses to majority-class predictions, while GlyphNet [11] achieves ROC-AUC 0.1180, reflecting the difficulty of adapting a single-image fraud classifier to a pairwise similarity task. Collectively, these results demonstrate that rendering-aware pairwise representations are substantially more effective for homoglyph detection than either string-based similarity measures or general-purpose semantic embeddings.

Table 6 summarises selected results reported in prior work for contextual reference. These results were not repro-

duced under identical conditions, with different datasets, train-test partitions, and experimental protocols, and therefore should not be interpreted as direct benchmarks. Nevertheless, the reported ROC-AUC values are broadly consistent with the findings of this paper. The Siamese-CNN model of Woodbridge et al. [7] reports ROC-AUC 0.97 on domain-name homoglyph detection, while Vinayakumar and Soman [30] report ROC-AUC values between 0.93 and 0.98 for Siamese CNN, LSTM, and GRU architectures. More recently, Jovanovic et al. [10] reported ROC-AUC 0.6874 for a zero-shot SigLIP representation and up to 0.98 after task-specific contrastive alignment and ensemble methods. Although these figures are not directly comparable to the Woodbridge test results reported here, they provide additional evidence that task-specific visual or sequential modeling is necessary for strong homoglyph detection performance and that generic semantic representations are insufficient.

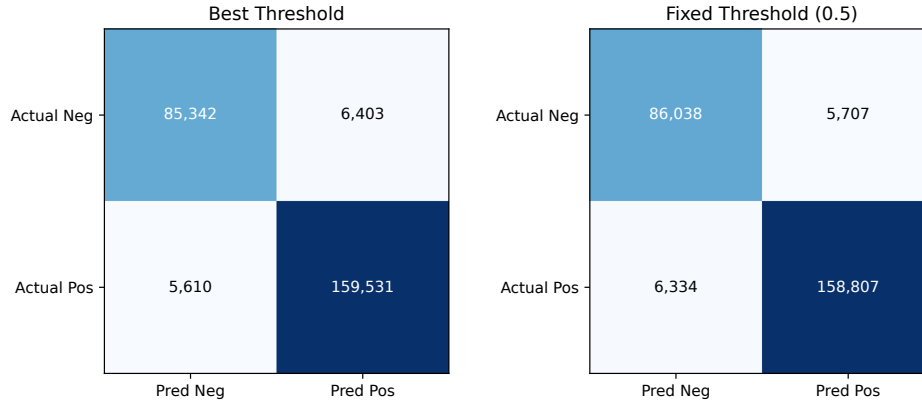


Figure 3: Test-set confusion matrices for the Conv1D+Transformer encoder (max pooling) at the best-F1 threshold ($t = 0.4557$, left) and at 0.5 (right).

Table 4: Test-set performance of the three encoder variants. All models use the same preprocessing configuration ($W = 6$, $S = 6$, mean pooling at sweep stage, seed 7, 20 epochs). Threshold-dependent metrics are reported at the best-F1 operating point. The Transformer uses max pooling, selected from the 5-epoch pooling ablation. Best value in each column is shown in bold.

Encoder	Val AUC	Test AUC	Avg Prec	F1	MCC	Specificity
Conv1D (baseline)	0.9888	0.9677	0.9820	0.9309	0.8050	0.8679
BiLSTM	0.9933	0.9844	0.9906	0.9540	0.8695	0.9032
Transformer (max)	0.9965	0.9900	0.9943	0.9637	0.8980	0.9302

5. Discussion

Sequential encoding as the core driver. The progression Conv1D (0.9677) $\rightarrow$ BiLSTM (0.9844) $\rightarrow$ Transformer (0.9900) on test AUC closely mirrors the progression of model expressiveness. The Conv1D encoder processes each strip independently, capturing intra-strip texture but no inter-strip dependencies. The BiLSTM introduces a recurrent state that accumulates positional context as it scans the strip sequence, allowing the model to detect whether a suspicious character cluster occurs in a position that is consistent with a known substitution pattern. The Transformer further replaces this sequential bottleneck with global self-attention and Rotary Position Embeddings [28], enabling direct comparison between any two strips regardless of distance. The consistent ranking across all metrics—AUC, average precision, F1, MCC, and specificity—confirms that architectural expressiveness over the sequential dimension translates cleanly into discrimination quality.

Shrinking validation-to-test gap. The gap between validation and test AUC narrows monotonically across the three encoders: 0.0211 (Conv1D), 0.0089 (BiLSTM), and 0.0065 (Transformer). A larger gap would suggest overfit-

ting to the validation distribution, but the opposite pattern is observed here. One interpretation is that the more expressive models learn a richer, geometry-grounded representation of visual similarity that generalises more readily to unseen pairs. The smaller gap also provides practical reassurance that the reported test numbers are stable estimates of real-world performance rather than artefacts of a lucky validation split.

Max pooling and localized substitution detection. The Transformer is evaluated with max pooling, which reduces the strip-level embedding sequence to the single highest-activation element before the similarity head. This choice is well-motivated for the homoglyph domain: a fraudulent name typically differs from its legitimate counterpart in only one or two character positions. Max pooling ensures that the most discriminative strip—the one capturing the substituted glyph—dominates the final embedding, rather than being diluted by the many strips encoding unmodified characters. The ablation in Section confirmed that max pooling outperformed mean and attention pooling for the Transformer, supporting this interpretation.

Validation of the thesis framing. The fastText and GlyphNet results provide complementary validation of

Table 5: Test-set performance of all methods. Val AUC is reported only for the three trained encoders; baselines are threshold-free ranking methods. All encoder metrics are recorded at the best-F1 threshold. Best value in each column is shown in bold. The Δ row reports absolute improvements of the Transformer over Damerau–Levenshtein (the best string-distance baseline by Test AUC).

Method	Category	Val AUC	Test AUC	Avg Prec	Best-F1	MCC
Levenshtein	String distance	–	0.8793	0.9235	0.8569	0.5828
Damerau–Levenshtein	String distance	–	0.8869	0.9278	0.8638	0.5992
Token Set Ratio	String similarity	–	0.8425	0.9020	0.8359	0.4982
TypoPegging	Weighted edit distance	–	0.8806	0.9303	0.8578	0.5796
fastText embeddings	Semantic embedding	–	0.6519	0.7746	0.7826	0.0000
GlyphNet	Visual CNN (unpaired)	–	0.1180	0.4587	0.7826	0.0000
Conv1D (ours)	Visual-sequential	0.9888	0.9677	0.9820	0.9309	0.8050
BiLSTM (ours)	Visual-sequential	0.9933	0.9844	0.9906	0.9540	0.8695
Transformer (ours)	Visual-sequential	0.9965	0.9900	0.9943	0.9637	0.8980
Δ vs. best string baseline			+0.1031	+0.0665	+0.0999	+0.2988

Table 6: Selected results reported by other authors on homoglyph detection benchmarks, included for contextual reference only. $\dagger$ Results are taken as reported by the original authors on their respective datasets and evaluation splits; they have not been independently reproduced under the conditions used in Table 5 and are not directly comparable to those rows. SigLIP zero-shot additionally reports Accuracy 0.6083, Precision 0.7735, Recall 0.5525 as given in [?].

Method	Category	Reported ROC-AUC	Source
Siamese-CNN † (domain names)	Visual CNN (Siamese, reported)	0.97	[7]
Siamese-GRU †	Visual RNN (Siamese, reported)	0.98	[30]
Siamese-LSTM †	Visual RNN (Siamese, reported)	0.97	[30]
Siamese-CNN †	Visual RNN (Siamese, reported)	0.93	[30]
SigLIP zero-shot †	Semantic embedding (reported)	0.6874	[10]

the central thesis claim. fastText demonstrates that semantic proximity in language embedding space bears no relationship to visual similarity between rendered names; a system relying on word embeddings would fail to detect visually confusable pairs that are lexically distant. GlyphNet demonstrates that visual rendering is necessary but not sufficient: a model that classifies individual names as “suspicious” cannot recover pairwise similarity, because it has no mechanism to compare the two inputs against each other. Together, these two failure modes define the niche that the proposed Siamese visual-sequential architecture is designed to fill.

Limitations. Several limitations bound the generalisability of the current results. First, all rendering uses a single font (DejaVu Sans); glyphs that are visually confusable only in particular typefaces would not be captured, and the model may not transfer to multi-font scenarios without retraining or data augmentation. Second, all reported numbers derive from a single random seed (seed 7); while the val-to-test gap and baseline margins are large enough to be unlikely artefacts of seed variance, formal confidence

intervals are not provided. Third, the 74-run preprocessing sweep used an unseeded random train/validation split, so individual sweep results carry additional variance beyond that of the final trained models. Fourth, the evaluation is restricted to Latin-script names; extension to cross-script homoglyphs (e.g. Cyrillic or Greek characters that visually resemble Latin letters) would require a new dataset and may demand architectural changes to handle variable rendering characteristics across scripts [13].

6. Conclusion

Homoglyph spoofing remains an active and growing threat to users and organisations: Q1 2025 alone saw over one million phishing attacks recorded by APWG [1], many of which exploit the human visual system’s tendency to conflate characters that share similar pixel structure. Existing detection approaches each fail to address this threat at the right level of abstraction. String edit distances [4, 5] treat characters as discrete symbols and are blind to the visual appearance of the glyphs they represent. Semantic word

embeddings [8] encode distributional meaning rather than visual form and, as confirmed empirically here, collapse to majority-class predictions on this task. Prior image-based classifiers such as GlyphNet [11] operate on individual rendered names in isolation and have no mechanism for pairwise visual comparison. What is needed is a detection framework that renders names as images, encodes their appearance in a way that is sensitive to local character-level structure, and learns pairwise visual similarity end-to-end. This thesis has developed and evaluated such a framework.

Three contributions are reported. First, a principled preprocessing framework was established through a systematic 74-run sweep over slice width, stride, pooling strategy, and padding removal, with all sweep runs conducted for 20 epochs using the Conv1D baseline encoder. Each hyperparameter was selected based on its marginal distribution across all configurations in the sweep rather than by picking the single best-performing run. The selected configuration—mean pooling, boundary slices retained, $W = 6$ px, $S = 6$ px (nonoverlapping)—achieved the highest mean validation AUC of any width-stride pairing (0.7541) with the lowest cross-configuration variance ($\sigma = 0.0183$), and produces a sequence length sufficient for the sequential encoders to attend to individual character-width regions. The single best individual sweep run (mean pooling, $W = 32$, $S = 32$, no padding removal) achieved a validation AUC of 0.8127 but was not selected because a 32-pixel slice width collapses a typical rendered name into only three to six strips—too short a sequence for meaningful position-sensitive modeling by the BiLSTM and Transformer encoders. Training the selected configuration from scratch with seed 7 fixed for 20 epochs achieved a validation ROC-AUC of 0.9888 on the Conv1D encoder. The marginal-effect analysis confirmed that pooling strategy and slice width are the dominant design choices, providing principled guidance for future work on rendering-based text encoders.

Second, three sequential encoder variants were evaluated under controlled conditions — fixed seed, identical rendering configuration, and identical training budget — enabling the first systematic comparison of Conv1D, BiLSTM, and Transformer architectures for the visual homograph detection task. The Conv1D encoder, which captures only local inter-strip context, achieved a test ROC-AUC of 0.9677. The BiLSTM, which integrates a recurrent state over the full strip sequence, improved this to 0.9844. The Transformer with max pooling and Rotary Position Embeddings [28], which applies global self-attention across all strip positions simultaneously, achieved the highest test ROC-AUC of 0.9900. The consistent ranking across all reported metrics, including AUC, average precision, F1, MCC, and specificity, establishes the Transformer as the strongest architecture for this task under the conditions studied.

The best-performing model substantially outperforms all string-based baselines: the Transformer’s test AUC of 0.9900 exceeds Damerau-Levenshtein (0.8869), TypoPegging (0.8806), Levenshtein (0.8793), and Token Set Ratio (0.8425) by 8.1–10.3 percentage points. The semantic embedding negative control (fastText, AUC 0.6519) confirms that distributional language representations provide no discriminative signal on this task. The closest published competitor is the VA-TE ensemble of Jovanovic-Hacon et al. [10], which reports a ROC-AUC of 0.9800 by projecting frozen vision-language model text encodings through a contrastive projection head. The Transformer proposed here achieves a test ROC-AUC of 0.9900, exceeding VA-TE’s reported 0.9800; both results are evaluated on the Woodbridge dataset, making this a direct comparison on a common benchmark. The Transformer additionally requires no large pretrained backbone and produces slice-level embeddings that naturally support future hit-zone localization.

Two novelty claims are made. First, the vertical strip-slicing sequential framing in which a rendered name image is decomposed into an ordered sequence of column strips and encoded by a learned sequence model, is, to the author’s knowledge, a new approach to homograph detection. Second, the three-way encoder comparison (Conv1D, BiLSTM, Transformer) conducted under identical preprocessing and training conditions on the same dataset is the first such systematic study in the homograph detection literature.

The limitations noted above, namely single font rendering, a single random seed, and evaluation restricted to Latin script pairs, point directly toward future directions, including multi font training, multi seed confidence estimation, and cross script generalization, which are addressed in the following section.

6.1. Future Work

6.1.1. Hit-Zone Visualization and Evaluation

Although the `HitZoneModule` is fully implemented and integrated into the model pipeline, but its cross-attention weight maps have not yet been systematically evaluated. Future work should determine whether attention peaks correspond to known substitution locations in deceptive homograph pairs. For example, a `paypal.com` / `paypal.com` pair should produce a hot column at the position of the final 1/1 substitution rather than at arbitrary positions. Beyond qualitative inspection, a quantitative evaluation protocol is needed: given ground-truth character-level substitution annotations for a held-out subset of the test set, one can compute whether the attention peak falls within a tolerance window of the substituted character, yielding a weakly supervised localization accuracy analogous to the pointing-game metric used in visual

attribution benchmarks. A second axis of quantitative evaluation concerns attribution method: raw cross-attention weights aggregate information from a single layer, but attention rollout [20] — which recursively multiplies augmented attention matrices across Transformer layers to account for residual connections — has been shown to substantially improve correlation with input-token importance relative to raw weights [20]. Applying attention rollout within the HitZoneModule, and comparing the resulting attributions to raw weights using the localization accuracy metric above, would follow the pairwise visualization framework of Black et al. [21] and determine whether the additional computation is warranted. Completing this evaluation would transform the hitzone module from an architectural component into a validated interpretability tool suitable for deployment in analyst-facing review workflows.

6.1.2. Architectural Extensions

Several architectural directions were considered but not pursued within the scope of this thesis. First, the HitZoneModule currently uses a single `nn.MultiheadAttention` layer to compute cross-attention between the two slice sequences; extending to stacked cross-attention blocks with residual connections and layer normalization, following the encoder-decoder design of the original Transformer [26], could improve alignment quality for longer names where the relevant substitution may be buried among many unmodified characters. Second, the Transformer encoder uses a fixed two-layer depth and four-head configuration chosen for comparability with the BiLSTM; a depth and head count sweep analogous to the Conv1D preprocessing sweep would establish whether these are near-optimal or whether shallower or wider configurations perform comparably at lower computational cost. Third, the CLS-token pooling strategy, which prepends a learnable classification token to the slice sequence in the style of BERT [9], achieved a validation AUC of only 0.8571 after five training epochs, substantially behind max pooling’s 0.9919 over the same period. It is worth investigating whether this gap persists with longer training or whether it reflects a structural incompatibility: in language model pre-training, the CLS token benefits from a rich pre-training signal that primes it to aggregate sequence-level information, but in the present setting the model is trained from scratch on relatively short fixed-alphabet sequences, and the CLS token must learn its aggregation role entirely from the supervised contrastive objective without such priming. Fourth, while Rotary Position Embeddings [28] are already used in the Transformer encoder, a controlled ablation comparing RoPE against no positional encoding and against sinusoidal absolute encodings [27] would directly

test the hypothesis that relative position sensitivity is more informative than absolute indexing for variable-length name sequences — a hypothesis that is theoretically well-motivated but not empirically verified in this work.

6.1.3. Preprocessing and Rendering

The preprocessing sweep covered six slice widths, two stride conditions, two padding conditions, and two background colours, but held several rendering parameters fixed. All experiments used a single font (DejaVu Sans), a fixed rendering height of 32 pixels, and a black background. Future work should first verify that the black-background choice is not a confound: white-background rendering is standard in most document imaging contexts, and it is possible that the model has learned edge features specific to the white-on-black polarity. A simple transfer experiment involving re-evaluating the best checkpoint on white-background-rendered test pairs would determine whether the performance gap is real or artefactual. Multi-font training is a more ambitious extension: rendering each name in a randomly sampled font drawn from a diverse font pool at every epoch would force the model to learn character-level visual features that are invariant to font style rather than font-specific textures, addressing the font-bias concern raised explicitly by Lee et al. [15] as a fundamental limitation of fixed-font approaches. Additionally, the 32 pixel rendering height may be insufficient to resolve sub pixel stroke differences for narrow characters such as 1, l, and I. These characters are among the most commonly substituted glyphs in homoglyph attacks, and their distinguishing features, including serif presence, crossbar placement, and relative stroke width, may be lost at low resolution. An ablation over rendering heights (32, 48, 64 pixels) would test this hypothesis. Finally, the effect of anti-aliasing and subpixel rendering warrants examination: both alter the effective width of thin character strokes at small sizes and could meaningfully change the strip-level feature distribution that the convolutional stem learns.

6.1.4. Dataset Quality and Error Analysis

The dataset used in this work consists of domain name and process name pairs from Woodbridge et al. [7], with spoof pairs generated by automated character substitution scripts. A structured error analysis involving partitioning false positives and false negatives by string length, substitution type (single vs. multi-character, Latin vs. non-Latin character replacement), and character position (prefix, suffix, interior) would identify where each encoder fails and provide a principled basis for targeted data augmentation or hard negative mining strategies. It is also unclear whether the learned representations generalise beyond domain and process names to other spoofing contexts such as brand

names, business names, and usernames, which may exhibit different length distributions, character set statistics, and substitution patterns. A cross-dataset evaluation on the synthetic cross-script evaluation set constructed by Jovanovic-Hacon et al. [10]— which covers English, Italian, brand names, and usernames — or on the GlyphNet dataset [11] would provide a direct test of cross-domain generalisation without requiring new data collection.

6.1.5. Cross-Script Generalization

All experiments in this thesis use Latin-script name pairs. The hit-zone approach is in principle script-agnostic since the rendering and slicing pipeline imposes no constraints on the Unicode content of the input string, and the learned encoder operates on pixel arrays rather than character identities. However, the model has been trained exclusively on Latin-script data and has never encountered non-Latin glyphs at training time. Fine-tuning on, or zero-shot evaluation against, pairs involving Cyrillic, Greek, Arabic, and CJK characters would test whether the visual features learned for Latin homoglyphs transfer to other scripts. This is of direct practical importance: the most dangerous homoglyph attacks in the wild exploit precisely the characters that most closely mimic Latin letters at the pixel level, such as Cyrillic (U+0430) for Latin a, Greek (U+03BF) for Latin o, and the full family of visually confusable Unicode pairs documented by Roshanbin and Miller [3] and exploited by Struppek et al. [13]. Because these attacks succeed precisely because the substituted character is pixel-level indistinguishable from its Latin counterpart, a rendering-aware model that achieves high discrimination on Latin data is well-positioned to generalise, but the cross-script domain shift in glyph proportions, stroke widths, and background statistics may require fine-tuning on a small number of cross-script pairs to close any residual gap.

6.1.6. Efficiency and Deployment

The Transformer encoder achieves the best test AUC but at higher inference cost than the Conv1D and BiLSTM variants due to its $O(N^2)$ self-attention over the strip sequence. Three efficiency directions are worth pursuing. First, knowledge distillation from the Transformer into a smaller Conv1D or BiLSTM student model where the student is trained to match the teacher’s embedding space rather than just its binary predictions, following the embedding-level distillation objective could recover much of the Transformer’s representational quality at a fraction of the inference cost; given that the BiLSTM already closes 75% of the gap between Conv1D and Transformer, the distillation target is an attainable one. Second, for deployment against a fixed whitelist of legitimate names (e.g., a corporate brand registry or a DNS allowlist), all whitelist

embeddings can be precomputed and indexed offline, reducing inference to a single forward pass for the query name followed by a nearest-neighbour lookup against the cached embeddings — an approach analogous to the KD-tree retrieval used by Woodbridge et al. [7] but operating in the learned embedding space rather than pixel space. Third, the Conv1D stem accounts for approximately 123K of the model’s roughly 250K total parameters and dominates inference latency for short sequences; structured pruning or INT8 quantization of the stem weights represents a low-effort path to latency reduction that would primarily affect the simpler Conv1D encoder variant while preserving the full expressiveness of the sequential encoder that follows it.

The central open challenge in fine-grained visual homoglyph detection is that the discriminative signal is almost imperceptible: the pixel differences between a substituted homoglyph pair can be sub-pixel in width, and the attack succeeds precisely because casual human visual attention does not resolve them under normal reading conditions [2]. A detection system must therefore be more sensitive to these differences than the human perceptual system being targeted, while remaining fast enough for real-time deployment. The combination of sequential visual encoding, which grounds discrimination in rendered pixel structure rather than symbolic distance, and hit-zone localization, which makes the model’s evidence transparent to human reviewers, positions this line of work to address that challenge in a principled and deployable way. Cross-script robustness is the remaining frontier: closing the generalization gap to non-Latin scripts would extend the reach of the approach to the full scope of Unicode-based homoglyph attacks observed in practice.

References

- [1] Anti-Phishing Working Group. *Phishing Activity Trends Report: 1st Quarter 2025*. Tech. rep. Anti-Phishing Working Group, 2025. URL: https://docs.apwg.org/reports/apwg_trends_report_q1_2025.pdf.
- [2] R. Dhamija, J. D. Tygar, and M. Hearst. “Why phishing works.” en. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Montréal Québec Canada: ACM, Apr. 2006, pp. 581–590. ISBN: 9781595933720. DOI: 10.1145/1124772.1124861. URL: <https://dl.acm.org/doi/10.1145/1124772.1124861> (visited on 03/31/2026).
- [3] N. Roshanbin and J. Miller. “Finding Homoglyphs - A Step towards Detecting UnicodeBased Visual Spoofing Attacks.” en. In: *Web Information System Engineering – WISE 2011*. Ed. by A. Bouguettaya, M. Hauswirth, and L. Liu. Berlin, Heidelberg: Springer, 2011, pp. 1–14. ISBN: 9783642244346. DOI: 10.1007/978-3-642-24434-6_1.
- [4] V. I. Levenshtein. “Binary Codes Capable of Correcting Deletions, Insertions and Reversals.” en. *Soviet Physics Doklady*, 10, Feb. 1966, p. 707. URL: <https://ui.adsabs.harvard.edu/abs/1966SPhD...10..707L/abstract> (visited on 03/31/2026).
- [5] G. V. Bard. “Spelling-Error Tolerant, Order-Independent Pass-Phrases via the Damerau-Levenshtein String-Edit Distance Metric.” *IACR Cryptol. ePrint Arch.*, **2006**, 2007, p. 364. URL: <https://api.semanticscholar.org/CorpusID:16089150>.
- [6] T. Liu, Y. Zhang, J. Shi, Y. Jing, Q. Li, and L. Guo. “Towards quantifying visual similarity of domain names for combating typosquatting abuse.” In: *MILCOM 2016 - 2016 IEEE Military Communications Conference*. ISSN: 2155-7586. Nov. 2016, pp. 770–775. DOI: 10.1109/MILCOM.2016.7795422. URL: <https://ieeexplore.ieee.org/document/7795422> (visited on 03/31/2026).
- [7] J. Woodbridge, H. S. Anderson, A. Ahuja, and D. Grant. “Detecting Homoglyph Attacks with a Siamese Neural Network.” In: *2018 IEEE Security and Privacy Workshops (SPW)*. May 2018, pp. 22–28. DOI: 10.1109/SPW.2018.00012. URL: <https://ieeexplore.ieee.org/abstract/document/8424628> (visited on 03/31/2026).
- [8] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean. “Distributed Representations of Words and Phrases and their Compositionality.” In: *Advances in Neural Information Processing Systems*. Vol. 26. Curran Associates, Inc., 2013. URL: <https://proceedings.neurips.cc/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html> (visited on 04/01/2026).
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by J. Burstein, C. Doran, and T. Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 4171–4186. DOI:10.18653/v1/N19-1423. URL: <https://aclanthology.org/N19-1423/> (visited on 04/01/2026).
- [10] A. Jovanovic-Hacon, S.-W. Luo, H. Kim, H. Boriel, R. Palacios, M. Wanderley, F. S. Beserra, and A. Gupta. “Visually-Aligned Text Embeddings via Vision-Language Models for Homoglyph Detection.” *IEEE Access*, 2025. Early version; publication details subject to finalization. DOI: 10.1109/ACCESS.2024.0429000.
- [11] A. Gupta, L. S. Tomar, and R. Garg. *GlyphNet: Homoglyph domains dataset and detection using attention-based Convolutional Neural Networks*. arXiv:2306.10392. June 2023. DOI: 10.48550/arXiv.2306.10392. URL: <http://arxiv.org/abs/2306.10392> (visited on 03/31/2026).
- [12] A. Ginsberg and C. Yu. “Rapid Homoglyph Prediction and Detection.” In: *2018 1st International Conference on Data Intelligence and Security (ICDIS)*. 2018, pp. 17–23. DOI: 10.1109/ICDIS.2018.00010.
- [13] L. Struppek, D. Hintersdorf, F. Friedrich, M. Br, P. Schramowski, and K. Kersting. “Exploiting Cultural Biases via Homoglyphs in Text-to-Image Synthesis.” *Journal of Artificial Intelligence Research*, **78**, Dec. 2023, pp. 1017–1068. issn: 1076-9757. DOI: 10.1613/jair.1.15388. URL: <http://www.jair.org/index.php/jair/article/view/15388> (visited on 04/01/2026).
- [14] G. R. Koch. “Siamese Neural Networks for One-Shot Image Recognition.” In: 2015. URL:

- <https://api.semanticscholar.org/CorpusID:13874643>.
- [15] J. S. Lee, G. P. D. Yam, and J. H. Chan. *Phish-GAN: Data Augmentation and Identification of Homoglyph Attacks*. arXiv:2006.13742. Sept. 2020. DOI: 10.48550/arXiv.2006.13742. URL: <http://arxiv.org/abs/2006.13742> (visited on 03/31/2026).
 - [16] B. Shi, X. Bai, and C. Yao. *An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition*. arXiv:1507.05717. July 2015. DOI: 10.48550/arXiv.1507.05717. URL: <http://arxiv.org/abs/1507.05717> (visited on 03/31/2026).
 - [17] Y. Tatsunami and M. Taki. “Sequencer: Deep LSTM for Image Classification.” en. *Advances in Neural Information Processing Systems*, **35**, Dec. 2022, pp. 38204–38217. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/f9d7d6c695bc983fcfb5b70a5fbdfd2fAbstract.html (visited on 03/31/2026).
 - [18] F.-J. Tsai, Y.-T. Peng, Y.-Y. Lin, C.-C. Tsai, and C.-W. Lin. “Stripformer: Strip Transformer for Fast Image Deblurring.” en. In: *Computer Vision – ECCV 2022*. Ed. by S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, and T. Hassner. Cham: Springer Nature Switzerland, 2022, pp. 146–162. ISBN: 9783031198007. DOI: 10.1007/978-3-031-19800-7_9.
 - [19] F. Schroff, D. Kalenichenko, and J. Philbin. “FaceNet: A Unified Embedding for Face Recognition and Clustering.” In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 815–823. URL: https://www.cvfoundation.org/openaccess/content_cvpr_2015/html/Schroff_FaceNet_A_Unified_2015_CVPR_paper.html.
 - [20] S. Abnar and W. Zuidema. Quantifying Attention Flow in Transformers. en. May 2020. URL: <https://arxiv.org/abs/2005.00928v2> (visited on 04/01/2026).
 - [21] S. Black, A. Stylianou, R. Pless, and R. Souvenir. “Visualizing Paired Image Similarity in Transformer Networks.” en. In: 2022, pp. 3164–3173. URL: https://openaccess.thecvf.com/content/WACV2022/html/Black_Visualizing_Paired_Image_Similarity_in_Transformer_Networks_WACV_2022_paper.html (visited on 03/31/2026).
 - [22] A. Stylianou, R. Souvenir, and R. Pless. “Visualizing Deep Similarity Networks.” In: *2019 IEEE Winter Conference on Applications of Computer Vision (WACV)*. ISSN: 1550-5790. Jan. 2019, pp. 2029–2037. DOI: 10.1109/WACV.2019.00220. URL: <https://ieeexplore.ieee.org/abstract/document/8659098> (visited on 03/31/2026).
 - [23] E. U. H. Qazi, A. Almorjan, and T. Zia. “A One-Dimensional Convolutional Neural Network (1D-CNN) Based Deep Learning System for Network Intrusion Detection.” en. *Applied Sciences*, **12**(16), Aug. 2022, p. 7986. ISSN: 2076-3417. DOI: 10.3390/app12167986. URL: <https://www.mdpi.com/2076-3417/12/16/7986> (visited on 03/31/2026).
 - [24] S. Phiphitphatphaisit and O. Surinta. “Deep feature extraction technique based on Conv1D and LSTM network for food image recognition.” en. *Engineering and Applied Science Research*, **48**(5), July 2021, pp. 581–592. ISSN: 2539-6218. URL: <https://ph01.tcithaijo.org/index.php/easr/article/view/243559> (visited on 03/31/2026).
 - [25] B. S. Riza, R. Yunita, and R. Rosnelly. “Comparative Analysis of LSTM and BiLSTM in Image Detection Processing.” *J. Wirel. Mob. Networks Ubiquitous Comput. Dependable Appl.*, **15**(1), 2024, pp. 244–260. DOI: 10.58346/JOWUA.2024.I1.017. URL: <https://doi.org/10.58346/jowua.2024.i1.017>.
 - [26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. *Attention Is All You Need*. arXiv:1706.03762. Aug. 2023. DOI:10.48550/arXiv.1706.03762. URL: <http://arxiv.org/abs/1706.03762> (visited on 03/31/2026).
 - [27] P. Shaw, J. Uszkoreit, and A. Vaswani. “Self-Attention with Relative Position Representations.” In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. Ed. by M. Walker, H. Ji, and A. Stent. New Orleans, Louisiana: Association for Computational Linguistics, June 2018, pp. 464–468. DOI: 10.18653/v1/N18-2074. URL: <https://aclanthology.org/N18-2074/> (visited on 04/01/2026).

- [28] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864, Nov. 2023. DOI: 10.48550/arXiv.2104.09864. URL: <http://arxiv.org/abs/2104.09864> (visited on 04/01/2026).
- [29] T. Fawcett. “An introduction to ROC analysis.” *Pattern Recognition Letters*. ROC Analysis in Pattern Recognition, **27**(8), June 2006, pp. 861–874. issn: 0167-8655. DOI: 10.1016/j.patrec.2005.10.010. URL: <https://www.sciencedirect.com/science/article/pii/S016786550500303X> (visited on 04/01/2026).
- [30] V. R. and S. K.P. “Siamese neural network architecture for homoglyph attacks detection.” *ICT Express*, **6**(1), 2020, pp. 16–19. ISSN: 2405-9595. DOI: <https://doi.org/10.1016/j.icte.2019.05.002>. URL: <https://www.sciencedirect.com/science/article/pii/S2405959519300025>.